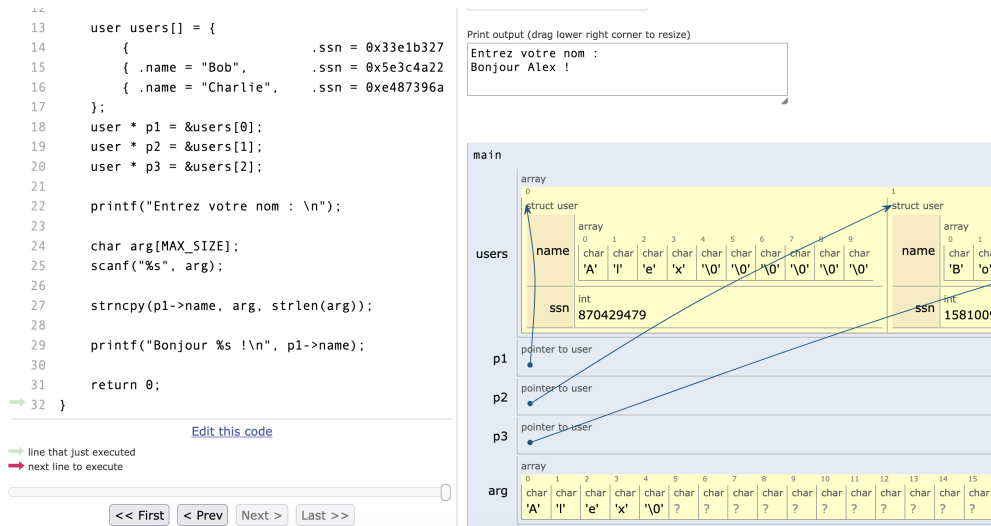


Exercice 1 : Buffer Overflow



```

13  user users[] = {
14      { .name = "Bob", .ssn = 0x33e1b327
15      { .name = "Charlie", .ssn = 0x5e3c4a22
16      { .name = "Charlie", .ssn = 0xe487396a
17  };
18  user * p1 = &users[0];
19  user * p2 = &users[1];
20  user * p3 = &users[2];
21
22  printf("Entrez votre nom : \n");
23
24  char arg[MAX_SIZE];
25  scanf("%s", arg);
26
27  strncpy(p1->name, arg, strlen(arg));
28
29  printf("Bonjour %s !\n", p1->name);
30
31  return 0;
32 }
  
```

Print output (drag lower right corner to resize)

Entrez votre nom :
Bonjour Alex !

main

array 0 struct user 1 struct user

users

name 0 char 1 char 2 char 3 char 4 char 5 char 6 char 7 char 8 char 9 char

ssn int 870429479 int 1581005

p1 pointer to user

p2 pointer to user

p3 pointer to user

arg array 0 char 1 char 2 char 3 char 4 char 5 char 6 char 7 char 8 char 9 char 10 char 11 char 12 char 13 char 14 char 15 char

'A' 'I' 'e' 'x' '\0' ? ? ? ? ? ? ? ? ? ? ? ? ? ?

- Ouvrez le lien CTutor suivant : [Exercice Buffer overflow](#)

Il contient le code présent dans le fichier `ex1.c`.

Pour cet exercice, vous n'avez pas le droit de modifier le code fourni. Vous devez seulement l'exécuter avec des entrées bien choisies.

- Exécutez le programme et vérifiez qu'en entrant votre nom, la fenêtre d'affichage (en haut à droite) affiche bien **Bonjour [votre nom]**.
- Examinez les instructions de saisie :

```

char arg[MAX_SIZE];
scanf("%s", arg);

strncpy(p1->name, arg, strlen(arg));
  
```

Que fait la fonction `strncpy` ? Quelle est la différence avec `strcpy` ?

Est-ce une utilisation sécurisée de `scanf` et de `strncpy` ?

- Quelle est la taille d'un objet structuré de type `user` ? Activez l'option **C/C++ details : Show memory addresses** en bas à droite.
- Tentez d'écrire plus de caractères que ne le permet le buffer de destination, et observez ce qu'il se passe.

L'objectif va être de réaliser une attaque par **buffer overflow** afin de découvrir le **nom** et le **numéro de sécurité sociale** de la personne qui se situe après nous dans la base de donnée.

Le pointeur **p1** pointe vers le **struct** contenant nos informations et que l'on va remplir. Nous souhaitons découvrir les informations de **p2**, uniquement en exécutant le programme avec des entrées bien choisies.

5. Trouver la bonne entrée à saisir au moment du **scanf** afin de voir apparaître le nom de l'utilisateur devant nous (ici, Bob) dans la fenêtre de texte de la sortie standard.

Justifiez votre raisonnement : pourquoi le nom est-il apparu ?

En particulier, comment la fonction **printf** détermine-t-elle la taille de la chaîne en entrée ?

6. Exécutez un buffer overflow similaire pour voir apparaître le **numéro de sécurité sociale de Bob**.

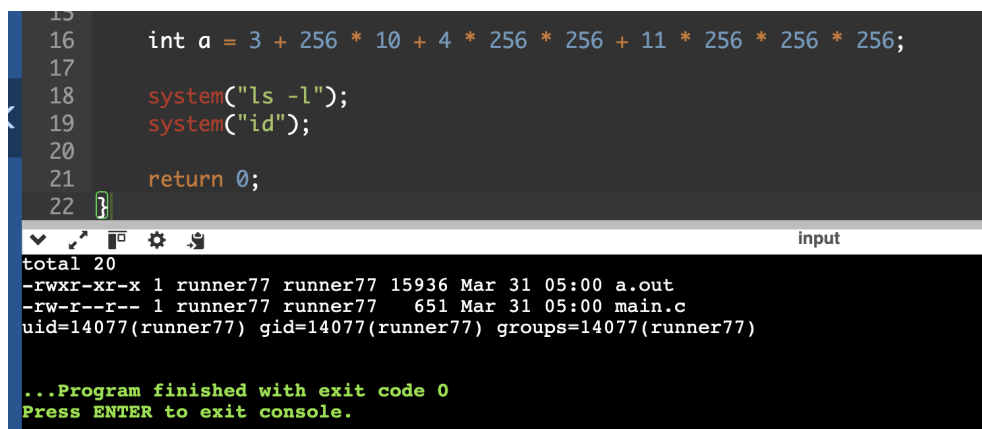
Celui-ci apparaîtra nécessairement **sous forme de caractères ASCII**.

Vous pouvez visiter la page [ascii-to-hex](#) pour convertir ces caractères en hexadécimal. Vérifiez votre réponse en lisant le code source de votre programme C.

Exercice 2 : Décompilation

1. Ouvrez la page [Online GDB](#). Sélectionnez le langage C en haut à droite.

Copiez le contenu du fichier **ex2.c** dans l'éditeur de Online GDB et exécutez le programme.



```
15
16     int a = 3 + 256 * 10 + 4 * 256 * 256 + 11 * 256 * 256 * 256;
17
18     system("ls -l");
19     system("id");
20
21     return 0;
22
```

total 20
-rwxr-xr-x 1 runner77 runner77 15936 Mar 31 05:00 a.out
-rw-r--r-- 1 runner77 runner77 651 Mar 31 05:00 main.c
uid=14077(runner77) gid=14077(runner77) groups=14077(runner77)

...Program finished with exit code 0
Press ENTER to exit console.

2. Quelles informations pouvez-vous déduire sur le terminal qui exécute votre code côté serveur ?

A quoi sert la fonction **system** ?

Comment s'appelle l'exécutable généré à partir de notre code source ?

3. Ajoutez à la suite des appels à **system** l'instruction suivante :

```
system("objdump --disassemble a.out");
```

Observez le résultat.

4. Pouvez-vous retrouver la valeur de la variable **a** dans le code ainsi décompilé ?

5. Pouvez-vous retrouver la chaîne de caractère `s` dans le code décompilé ?
6. Ajoutez les lignes suivantes après les appels à `system` :

```
system("echo \"\"");  
system("cat a.out");
```

Observer le résultat. Quelles informations pouvez-vous retrouver ?

Exercice 3 : Mémoire virtuelle

1. Observez le programme `ex3.c`. Compilez-le et exécutez-le.
2. Dans la console, exécutez le programme compilé avec la commande :

```
./main &
```

Où `main` est le nom de votre exécutable. Vous pouvez aller le chercher dans le dossier `cmake-build-debug`.

Apposer le symbole `&` permet d'effectuer une commande en **arrière-plan**.

3. Déterminez le **pid** (*Processus IDentifier*) du processus qui exécute votre exécutable, avec la commande `ps`.
4. Exécutez la commande suivante, pour afficher la **mémoire virtuelle** du processus en cours :

```
vmmap <pid>
```

En remplaçant `<pid>` par le pid de votre processus. Observez le résultat.

La *mémoire virtuelle* est une abstraction des ressources de stockage de la machine. C'est la vue sur la mémoire lors de l'exécution d'un programme. Ces adresses *virtuelles* sont traduites à la volée en *adresses réelles* pour correspondre aux emplacements réels.

5. Questions :

- A quoi correspondent les différents **REGION TYPE** ?
- Où sont stockées les chaînes de caractères `"Start"` et `"c = %d\nEnd\n"` dans la mémoire virtuelle ?
- Où est stocké le code compilé de l'exécutable dans la mémoire virtuelle (en cours d'exécution) ?
Indice : pour les deux dernières questions, regardez les permissions **PRT/MAX**.
- La pile est-elle **exécutable** ? (pouvons-nous tenter une attaque par injection de code ?)